



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: CAMPOS MEDRANO JOHN
BRANDON

N° de Cuenta: 418048324

GRUPO DE LABORATORIO: 2

GRUPO DE TEORÍA: 7

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12/09/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA 3

Modelado geométrico

1. Ejecución de los ejercicios

En esta práctica se construyeron dos figuras en tres dimensiones: un cohete espacial y una cruz formada por ocho pirámides. Ambas aparecen juntas en una sola ventana, de modo que se pueden observar con la misma cámara.

Se utilizó la estructura proporcionada en el laboratorio: Window, Camera, Shader, Mesh y Sphere. El cohete combina cilindros, conos, una esfera, cubos y una pirámide como aleta. La cruz reutiliza una pirámide completa en cuatro pares.

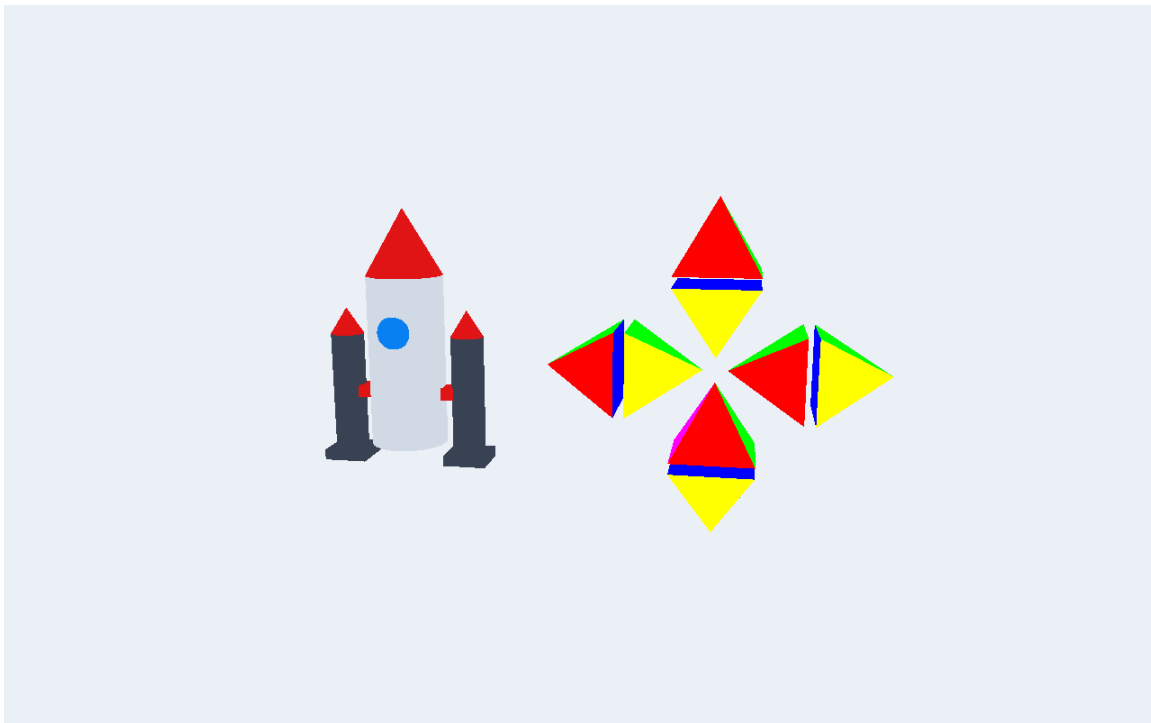


Figura 1. Ejecución final con el cohete a la izquierda y la cruz a la derecha.

La escena usa perspectiva y colores sólidos. No tiene un suelo negro ni una vista adicional con una pirámide aislada. Las piezas se revisan moviendo la cámara alrededor de las dos figuras.

1. Construcción del cohete y de la cruz

Cohete espacial

El cuerpo y los dos propulsores laterales son cilindros. La punta principal y las puntas laterales son conos. Una esfera aplanada forma la ventanilla; los cubos se usan como uniones y apoyos. Se conserva una sola pirámide como aleta trasera para incluir las cinco primitivas que pide el ejercicio.

P03-418048324.cpp | Código final

```
77 dibujar(cilindro,shader,escalar(.58f,2.6f,.58f),blanco);
78 dibujar(cono,shader,mover(0,1.8f,0)*escalar(.58f,1,.58f),rojo);
```

Figura 2. Fragmento que coloca el cuerpo cilíndrico y la punta del cohete.

Cruz de ocho pirámides

Se forman cuatro pares: arriba, abajo, izquierda y derecha. En cada par, una pirámide apunta en sentido contrario a la otra. Sus bases están separadas 0.16 unidades, un espacio pequeño que permite distinguir el azul al cambiar el ángulo de la cámara.

P03-418048324.cpp | Código final

```
68 const glm::mat4 pares[]={mover(0,centro,0),mover(0,-centro,0),
69 mover(-
    centro,0,0)*girar(90,{0,0,1}),mover(centro,0,0)*girar(90,{0,0,1})};
70 for(const auto& par:pares){
71 matriz(shader,posicion*par*mover(0,separacion/2,0));piramide.RenderMes
    hColor();
72 matriz(shader,posicion*par*mover(0,-
    separacion/2,0)*girar(180,{1,0,0}));piramide.RenderMeshColor();
73 }
74 }
```

Figura 3. Cuatro posiciones y dos dibujos por posición: ocho pirámides en la cruz.

Cada pirámide tiene cuatro caras triangulares: roja, verde, amarilla y magenta. La base cuadrada es azul y se dibuja con dos triángulos. Los vértices de cada cara se repiten para evitar que sus colores se mezclen con los de las caras vecinas.

1. Uso de las clases del laboratorio

Organización del programa

Window recibe el teclado, el ratón y la rueda. Camera calcula desde dónde se observa la escena y cómo se mueve la vista. Shader carga los archivos externos que aplican las transformaciones y los colores. Mesh y MeshColor dibujan las piezas; Sphere se usa para la ventanilla.

El main crea las piezas, coloca las dos figuras y llama a esas clases. Los controles ya no se implementan como callbacks dentro del main. En los shaders se habilitó la matriz view para que el movimiento de Camera se refleje en pantalla.

Window.cpp | Código final

```
30 void Window::ManejaMouse(GLFWwindow* w,double x,double y){
31     auto* self=static_cast<Window*>(glfwGetWindowUserPointer(w));
32     if(self->mouseFirstMoved){self->lastX=float(x);self->
        lastY=float(y);self->mouseFirstMoved=false;}
33     if(glfwGetMouseButton(w,GLFW_MOUSE_BUTTON_LEFT)==GLFW_PRESS){self->
        xChange+=float(x)-self->lastX;self->yChange+=self->lastY-float(y);}
34     self->lastX=float(x);self->lastY=float(y);
35 }
```

Figura 4. Manejo del ratón dentro de Window.cpp, como indicó el profesor.

P03-418048324.cpp | Código final

```
116     camera.keyControl(ventana.getKeys(),dt);camera.mouseControl(ventana.
        getXChange(),ventana.getYChange());camera.zoom(ventana.getScroll()*0.35
        f);
117     if(ventana.getAutomatic()){giro+=25*dt;camera.orbit(giro,18,14);}
118     if(test){const float ang[]={10,35,90,180,270,30,0,0};const float elev
        []={12,25,0,0,0,-40,65,-65};camera.orbit(ang[frame],elev[frame],14);}
```

Figura 5. El main consulta Window y utiliza Camera para mover la vista.

Controles

Las flechas permiten rodear las figuras. Al arrastrar con el botón izquierdo cambia la dirección de la mirada. W y S avanzan o retroceden; A y D desplazan a los lados; Q y E bajan o suben. La rueda acerca o aleja, Espacio activa el giro automático y R recupera la vista inicial.

1. Resultados desde otros ángulos

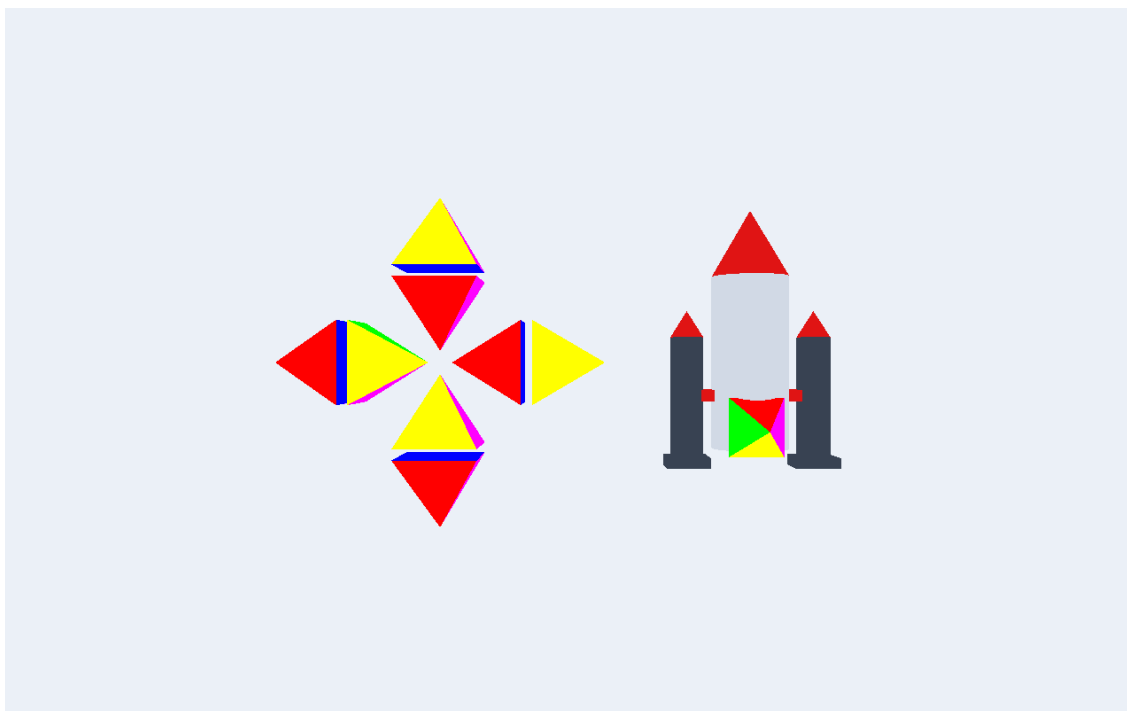


Figura 6. Vista posterior: se aprecia la única aleta piramidal del cohete.

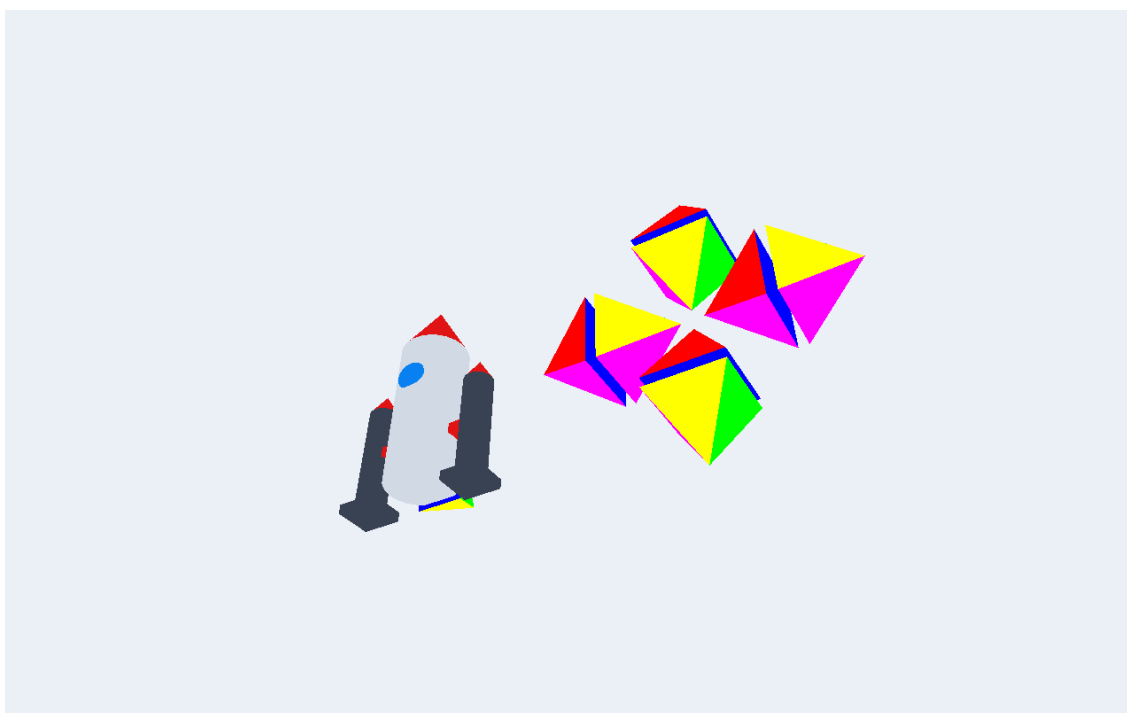


Figura 7. Vista inferior inclinada: las bases azules de las pirámides quedan visibles.

Las dos figuras siguen formando parte de la misma escena durante el recorrido. Desde algunos lados pueden superponerse visualmente, como ocurre al observar objetos reales alineados; basta cambiar el ángulo para distinguirlos.

2. Problemas encontrados y soluciones

Uso de la estructura proporcionada

La versión anterior del ejercicio de clase concentraba los controles en el main y no utilizaba todas las clases indicadas. Para este reporte se retomaron los archivos originales del laboratorio. El teclado y el ratón se manejan en Window, y la vista se calcula en Camera.

La cámara no se reflejaba en los shaders

Los shaders proporcionados tenían una línea que dibujaba sin aplicar view. Se corrigió para incluir proyección, vista y modelo. Así, al mover la cámara sí cambia el punto desde el cual se observan las figuras.

Caras de colores y base completa

Se ajustó el conteo de vértices de MeshColor para que coincida con la cantidad real de datos. La pirámide se dibuja con cuatro triángulos laterales y dos para la base. Esto permite mantener cinco caras completas y colores distintos.

Revisión de las bases azules

Al juntar totalmente dos pirámides por sus bases, el azul queda oculto. Se dejó un espacio pequeño entre ambas para poder revisar esa parte sin perder la forma de cada par.

Otros ajustes de funcionamiento

Se inicializaron las variables de Window y se añadió el retorno de su inicialización. También se corrigieron índices de Sphere que excedían los vértices disponibles. Los shaders se copian junto al ejecutable para que el programa los encuentre al abrirse fuera de Visual Studio.

La versión final se comprobó con capturas de ocho ángulos. Se verificó la ejecución en Release y Debug. Las imágenes de este reporte provienen del programa, no de un dibujo generado como referencia.

3. Conclusiones y bibliografía

La práctica muestra que se pueden construir objetos reconocibles combinando figuras básicas. En el cohete fue importante cambiar el tamaño y la posición de cada pieza. En la cruz, lo principal fue repetir una pirámide completa y revisar la orientación de los cuatro pares.

La dificultad fue media, sobre todo por el acomodo de las figuras y la revisión de sus caras. Observar únicamente el frente no era suficiente: mover la cámara permitió comprobar las bases azules y la aleta que queda detrás del cohete.

Usar las clases del laboratorio ayudó a separar las tareas del programa. Window se encarga de los controles, Camera de la vista y Shader de los archivos gráficos. Esta organización facilita localizar qué parte hay que cambiar cuando algo no funciona.

Como sugerencia de trabajo, conviene comprobar primero una pieza completa y después repetirla. También es útil compilar en Release y probar el ejecutable fuera de Visual Studio antes de preparar la entrega.

Créditos de implementación

Se utilizaron las clases y shaders proporcionados en el laboratorio. Para apoyar la corrección del código y la organización del reporte se utilizó una herramienta de inteligencia artificial; el resultado se verificó mediante la ejecución del programa.

Bibliografía

de Vries, J. (s. f.-a). Camera. LearnOpenGL.
<https://learnopengl.com/Getting-started/Camera>

de Vries, J. (s. f.-b). Transformations. LearnOpenGL.
<https://learnopengl.com/Getting-started/Transformations>